STAT 432 — Homework 08

Neel Nigam · nnigam2@illinois.edu · Fall 2026

Question 1: Boundary and margin

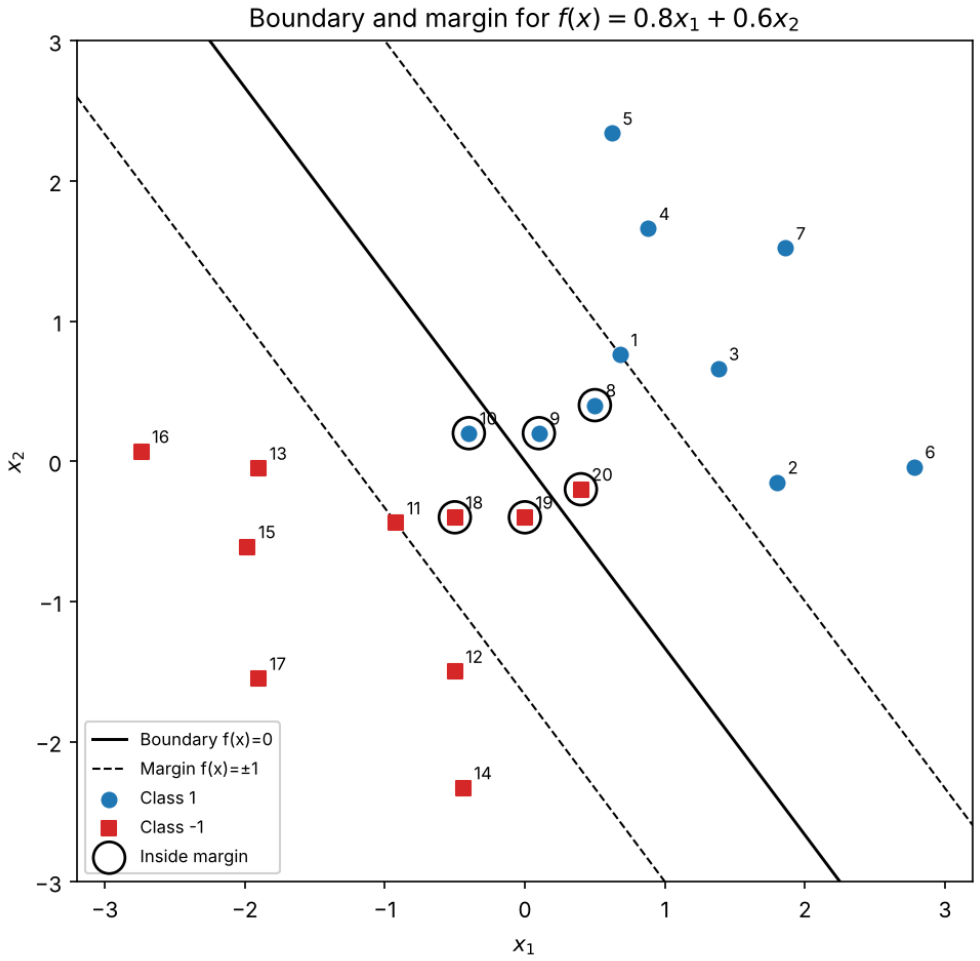
The score is $f(x) = 0.8x_1 + 0.6x_2$. Since $\|(0.8, 0.6)\|_2 = 1$, the distance from an observation to the boundary $f(x) = 0$ is $|f(x)|$. An observation is inside the margin when $yf(x) < 1$, on the margin when $yf(x) = 1$, and beyond the margin when $yf(x) > 1$. No new classifier was fitted.

(a) Distances and functional margins

Obs	x1	x2	y	f(x)	Distance	y·f(x)	Margin status	Misclassified
1	0.68	0.76	1	1.00	1.00	1.00	on	No
2	1.80	-0.15	1	1.35	1.35	1.35	beyond	No
3	1.38	0.66	1	1.50	1.50	1.50	beyond	No
4	0.88	1.66	1	1.70	1.70	1.70	beyond	No
5	0.62	2.34	1	1.90	1.90	1.90	beyond	No
6	2.78	-0.04	1	2.20	2.20	2.20	beyond	No
7	1.86	1.52	1	2.40	2.40	2.40	beyond	No
8	0.50	0.40	1	0.64	0.64	0.64	inside	No
9	0.10	0.20	1	0.20	0.20	0.20	inside	No
10	**-0.40**	**0.20**	**1**	**-0.20**	**0.20**	**-0.20**	**inside**	**Yes**
11	-0.92	-0.44	-1	-1.00	1.00	1.00	on	No
12	-0.50	-1.50	-1	-1.30	1.30	1.30	beyond	No
13	-1.90	-0.05	-1	-1.55	1.55	1.55	beyond	No
14	-0.44	-2.33	-1	-1.75	1.75	1.75	beyond	No
15	-1.98	-0.61	-1	-1.95	1.95	1.95	beyond	No
16	-2.74	0.07	-1	-2.15	2.15	2.15	beyond	No
17	-1.90	-1.55	-1	-2.45	2.45	2.45	beyond	No
18	-0.50	-0.40	-1	-0.64	0.64	0.64	inside	No
19	0.00	-0.40	-1	-0.24	0.24	0.24	inside	No
20	**0.40**	**-0.20**	**-1**	**0.20**	**0.20**	**-0.20**	**inside**	**Yes**

Shaded rows are inside the margin; bold rows are misclassified.

(b) Plot



Class 1 is shown as blue circles and class −1 as red squares. The solid line is the boundary $f(x) = 0$, the dashed lines are the margin lines $f(x) = \pm 1$, and the observations inside the margin are circled in black. Axes are $x_1 \in [-3.2, 3.2]$ and $x_2 \in [-3, 3]$ with an equal aspect ratio.

(c) Counts and misclassifications

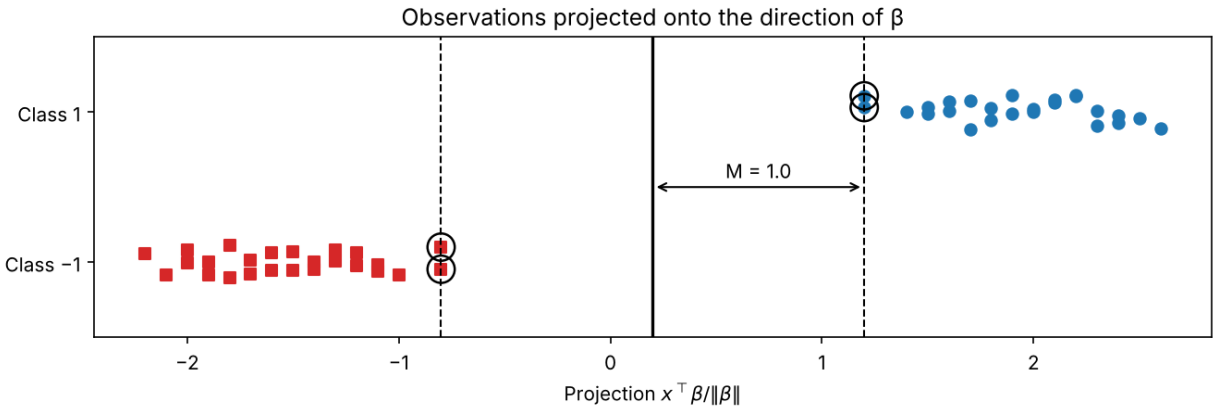
Inside the margin: 6 observations (8, 9, 10, 18, 19, 20).
On the margin: 2 observations (1, 11).
Beyond the margin: 12 observations.

Misclassified: observations 10 and 20. They are the only observations with $yf(x) < 0$: observation 10 has $y = 1$ but $f(x) = -0.20$, and observation 20 has $y = -1$ but $f(x) = 0.20$. Both lie inside the margin on the wrong side of the boundary. The other four observations inside the margin (8, 9, 18, 19) are correctly classified but have $0 < yf(x) < 1$.

Question 2: Margin in four dimensions

(a) Projection onto the direction of β

With $\boldsymbol{\beta} = (1, 2, -2, 4)^T$, $\|\boldsymbol{\beta}\| = \sqrt{(1 + 4 + 4 + 16)} = 5$. Each observation is projected onto the unit direction $\boldsymbol{\beta}/\|\boldsymbol{\beta}\|$, giving the position $\mathbf{x}^T\boldsymbol{\beta}/\|\boldsymbol{\beta}\|$. The boundary $f(\mathbf{x}) = -1 + \mathbf{x}^T\boldsymbol{\beta} = 0$ is the same as $\mathbf{x}^T\boldsymbol{\beta} = 1$, so it sits at position 1/5 = 0.2 on this axis.



Points are jittered vertically only. The solid line is the decision boundary, the dashed lines are at distance M from it, and the closest observations are circled.

Class −1 projections range from -2.2 to -0.8, and class 1 projections range from 1.2 to 2.6, so every observation is correctly classified. The closest observations are at -0.8 and 1.2, which are each $|-0.8 - 0.2| = |1.2 - 0.2| = 1.0$ from the boundary. Therefore the geometric margin is $\boldsymbol{M = 1}$. The closest observations are 4, 15, 33, 36, two from each class.

(b) SVM normalization

The geometric margin of observation i is $y_i f(\mathbf{x}_i)/\|\boldsymbol{\beta}\|$. Since $M = 1$ and $\|\boldsymbol{\beta}\| = 5$, the smallest functional margin is $\min_i y_i f(\mathbf{x}_i) = M\|\boldsymbol{\beta}\| = 5$. Dividing β_0 and $\boldsymbol{\beta}$ by 5 leaves the boundary unchanged and sets this smallest functional margin to 1:

$$\boldsymbol{\beta} = (0.2, 0.4, -0.4, 0.8)^T, \beta_0 = -0.2.$$

Check: the rescaled $\|\boldsymbol{\beta}\| = 1$, so $M = 1/\|\boldsymbol{\beta}\| = 1$, matching part (a).

Question 3: Peer review of a Homework 7 skill

GitHub Issue: https://github.com/tgeorg23/stat432-fall2026/issues/2

Peer review: tgeorg23, Homework 7 LDA/QDA skill

Overall assessment. The skill is clear, generic and careful about leakage. It seals the test set, keeps CV folds fixed across candidates, fits preprocessing inside each fold, uses the test set once, and tells the agent not to fabricate results. I copied SKILL.md unchanged (3,756 bytes, matching the stated size) and gave my agent only the data path and the Homework 7 coding. It produced a valid one-page analysis without any steering. It finished in about 1.5 minutes, so I did not stop it early. The protocol worked as intended: no test data was used before the final evaluation, and the limitations were honest. My main concerns are that too many key choices are left to the agent or the prompt, that the selection rule overreacts to noise, and that the test result in the report lacks context.

1. The skill leaves key protocol settings unspecified, so the report reflects the prompt more than the skill.

Observed: The author's invocation prompt supplied the test fraction (25%), the seed (2026), the fold count (5), and the candidate set (PCA with 2–8 components, no blends). The skill gives no defaults for any of these. With only the data information, my agent chose seed 432, 10-fold CV, PCA with 1–8 components, and nine LDA/QDA probability blends. That gave 27 candidates and a different selected model: PCA(7)+LDA, CV 0.254, test 0.267, versus PCA(8)+LDA, CV 0.272, test 0.241 in the report.

Why it matters: The assignment asks the skill to direct the analysis. When a prompt has to spell out the method, the skill is not reusable, and two users running it will get different analyses and possibly different conclusions. These settings are not dataset-specific, so they belong in the skill.

Recommendation: Add defaults that the prompt can override. For example: a 25% stratified test set, 5- or 10-fold stratified CV, a stated default seed, PCA with k = 1 to p−1 components, and blend weights 0 to 1 in steps of 0.1. Also say whether blends are tried by default.

2. The "lowest mean CV error" rule picks models whose advantage is within CV noise.

Observed: In the report, PCA-8+LDA (0.2717) beat plain LDA (0.2860). The fold SDs were about 0.04–0.05 over 5 folds, so the standard error is about 0.02 and the gap of 0.014 is below it. In my run, PCA(7)+LDA (0.254) beat LDA (0.266) with a CV SE of about 0.025. In both cases the "winner" is a PCA step that keeps almost every component. Standardized PCA with all 9 components followed by LDA gives the same classifier as plain LDA, so PCA-8 drops only one direction. That adds a preprocessing step for a gain that is not real. Ties are broken by SD and then simplicity, but exact ties essentially never happen, so the simplicity tie-break never applies.

Why it matters: The selected model is more complex than the evidence supports, and the report's "best" model is mostly an artifact of fold noise. The author's limitations paragraph says the minimum CV error is optimistic, but it does not say that the top candidates cannot be told apart.

Recommendation: Use a one-standard-error rule. Among the candidates within one CV SE of the minimum, choose the simplest, and define "simplest": for example, plain LDA, then plain QDA, then fewer PCA components, then blends. Require the report to state when the selected model is not clearly better than plain LDA or QDA.

3. The single test error is reported without context.

Observed: The report's table gives a test error only for the selected model (0.2414). It also shows a leftover pandas index column (0–3) as the first column. The test error is below every CV estimate, which the reader has to interpret alone. The skill allows baseline test errors in the same final pass, but it does not require them, and it does not ask for a no-information baseline. My run did report these: LDA and QDA both had a test error of 0.276, and always predicting "no CHD" gave 0.345. Those numbers made it clear that the selected model differs from LDA by one test observation and is only moderately better than the majority class.

Why it matters: With 116 test observations, the test error has an SE of about 0.04. Without baselines or an uncertainty statement, 0.2414 looks like a meaningful improvement when it is within sampling noise.

Recommendation: Have the skill require, in the same single final pass, test errors for ordinary LDA and QDA, the majority-class error rate, and an approximate SE or interval for the test error. In the report, remove the index column from the table and add those comparison rows.

Appendix: Python code for Questions 1 and 2

```
"""STAT 432 Homework 08 — Questions 1 and 2 (margin geometry)."""
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ---------------------------------------------------------------- Question 1
d = pd.read_csv("data/margin-geometry-20.csv")
w = np.array([0.8, 0.6])                       # ||w|| = 1
d["f"] = d[["x1", "x2"]].to_numpy() @ w        # score
d["distance"] = d["f"].abs() / np.linalg.norm(w)  # distance to f(x)=0
d["yf"] = d["y"] * d["f"]                      # functional margin
d["status"] = np.select(
    [np.isclose(d["yf"], 1), d["yf"] < 1], ["on", "inside"], default="beyond"
)
d["misclassified"] = d["yf"] < 0

table = d[["observation", "x1", "x2", "y", "f", "distance", "yf", "status", "misclassified"]]
print(table.round(3).to_string(index=False))
print("\nCounts:\n", d["status"].value_counts().reindex(["inside", "on", "beyond"]))
print("Misclassified observations:", d.loc[d["misclassified"], "observation"].tolist())

# Plot
fig, ax = plt.subplots(figsize=(7, 6.5))
x1 = np.linspace(-3.2, 3.2, 200)
for c, ls, lab in [(0, "-", "Boundary f(x)=0"), (1, "--", "Margin f(x)=±1"), (-1, "--", None)]:
    ax.plot(x1, (c - 0.8 * x1) / 0.6, color="black", ls=ls, lw=1.4 if c == 0 else 1, label=lab)
for cls, col, mk in [(1, "tab:blue", "o"), (-1, "tab:red", "s")]:
    s = d[d["y"] == cls]
    ax.scatter(s["x1"], s["x2"], color=col, marker=mk, s=45, zorder=3, label=f"Class {cls}")
inside = d[d["status"] == "inside"]
ax.scatter(inside["x1"], inside["x2"], s=220, facecolors="none", edgecolors="black",
           lw=1.3, zorder=4, label="Inside margin")
for _, r in d.iterrows():
    ax.annotate(int(r["observation"]), (r["x1"], r["x2"]), xytext=(5, 4),
                textcoords="offset points", fontsize=8)
ax.set_xlim(-3.2, 3.2); ax.set_ylim(-3, 3); ax.set_aspect("equal")
ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
ax.set_title(r"Boundary and margin for $f(x)=0.8x_1+0.6x_2$")
ax.legend(loc="lower left", fontsize=8, framealpha=0.9)
fig.tight_layout(); fig.savefig("q1-margin-plot.png", dpi=150)

# ---------------------------------------------------------------- Question 2
e = pd.read_csv("data/margin-geometry-4d.csv")
beta0, beta = -1.0, np.array([1.0, 2.0, -2.0, 4.0])
X = e[["x1", "x2", "x3", "x4"]].to_numpy()
norm_b = np.linalg.norm(beta)                  # = 5
proj = X @ beta / norm_b                       # position along unit direction beta/||beta||
boundary = -beta0 / norm_b                     # f = 0  <=>  proj = 0.2
yf = e["y"].to_numpy() * (beta0 + X @ beta)
geo = yf / norm_b                              # signed geometric margins
M = geo.min()
print(f"\n||beta|| = {norm_b}, boundary at projection {boundary}")
print(f"All correctly classified: {(yf > 0).all()}")
print(f"Geometric margin M = {M:.3f}; closest obs: {e.loc[np.isclose(geo, M), 'observation'].tolist()}")
print(f"Smallest functional margin = {yf.min():.3f}")
print(f"Normalized: beta = {beta / yf.min()}, beta0 = {beta0 / yf.min()}")

rng = np.random.default_rng(1)
ypos = np.where(e["y"] == 1, 1.0, 0.0) + rng.uniform(-0.12, 0.12, len(e))  # vertical jitter only
fig, ax = plt.subplots(figsize=(9, 3.2))
for cls, col, mk in [(1, "tab:blue", "o"), (-1, "tab:red", "s")]:
    m = e["y"].to_numpy() == cls
    ax.scatter(proj[m], ypos[m], color=col, marker=mk, s=35, label=f"Class {cls}")
closest = np.isclose(geo, M)
ax.scatter(proj[closest], ypos[closest], s=200,
           facecolors="none", edgecolors="black", lw=1.2)
ax.axvline(boundary, color="black", lw=1.6, label="Boundary (0.2)")
for v in (boundary - M, boundary + M):
    ax.axvline(v, color="black", ls="--", lw=1)
ax.annotate("", xy=(boundary + M, 0.5), xytext=(boundary, 0.5),
            arrowprops=dict(arrowstyle="<->"))
```

```
ax.text(boundary + M / 2, 0.56, f"M = {M:.1f}", ha="center")
ax.set_yticks([0, 1], ["Class −1", "Class 1"]); ax.set_ylim(-0.5, 1.5)
ax.set_xlabel(r"Projection $x^\top\beta/\|\beta\|$")
ax.set_title("Observations projected onto the direction of β")
fig.tight_layout(); fig.savefig("q2-projection-plot.png", dpi=150)
```